BACHELORARBEIT im Studiengang

Wirtschaftsinformatik – Bachelor of Science (B.Sc.)

aus dem Fach/Modul: Wirtschaftsinformatik

gestellt von: Professor Dr. Peter Fettke

mit dem Thema: Prozessvorhersage mittels Transformer-Netzen am Beispiel der Fischertechnik APS

bearbeitet von

Name: Nicolas Christian Edelmann

Adresse: Reinhold-Zeller-Str. 19, 66386 St. Ingbert

Abgabetermin: .....

Spätester Beurteilungstermin: .....

Eidesstattliche Erklärung

Hiermit versichere an Eides statt, dass ich diese Arbeit selbständig angefertigt und keine anderen als die angegebenen Hilfsmittel verwendet habe. Alle wörtlichen oder sinngemäßen Entlehnungen sind deutlich als Solche gekennzeichnet.

Saarbrücken, June 11, 2026 _____

Nicolas Christian Edelmann

Contents

List of Figures

1

Introduction

Traditionally discovering knowledge about your business processes required combining estimates and manual data collection. This acquired knowledge would then be used to analyse, redesign and optimise said processes or to make process-external decisions (Di Francescomarino et al., 2026). This type of knowledge discovery is naturally error-prone and tends to be time-consuming.

Nowadays, more and more businesses rely on enterprise systems to store highly detailed process execution information, based on sensoric outputs, workflow data and machine logs. By applying various process mining techniques to these datasets, crucial knowledge can be extracted (Aalst, 2016), which can then be used in multiple business process lifecycles (Muehlen & Ho, 2006). During process *design*, analysis of historical data can provide comparison points, during process *monitoring* outliers and bottlenecks can be detected.

A subfield of process monitoring is predictive process monitoring. It is focused on predicting the behaviour of process executions under certain conditions. It can then provide information about the business goals, such as expected failure rates or execution time. If a process is structured into multiple events, *next event prediction* is concerned about the next events activity or surrounding metadata. Here, machine learning approaches have been on the rise, from using LSTMs (Camargo et al., 2019; Tax et al., 2017) to modified transformer architectures (Bukhsh et al., 2021; Ni et al., 2023).

In modern environments, process executions are inherently concurrent and parallel. As a result, given a running process execution, **the** next event is not clearly defined: Due to the parallel execution multiple events could be executed at the same time, or in an arbitrary order depending on system load. While a linear log collected by an enterprise system might exist, due to the parallel execution this order should not be fully enforced onto predictions. As an example, if two machines need to produce A and B, and then a third machine combines them, the imaginary prediction trace

$\texttt{Produce}\ B \rightarrow \texttt{Produce}\ A \rightarrow \texttt{Combine}\ A \wedge B$

does not necessarily conflict with

$$\texttt{Produce}\, A \rightarrow \texttt{Produce}\, B \rightarrow \texttt{Combine}\, A \wedge B$$

However there does exist a partial order of events within a process due to causal relationships between the events. Following the example from before, clearly

$$\texttt{Combine}\, A \wedge B \rightarrow \texttt{Produce}\, A \rightarrow \texttt{Produce}\, B$$

is not feasable, as one can not combine non-existing products.

In this work, we present an approach to use transformer networks to solve the next event prediction problem, while working around the concurrency uncertainty by modelling it using the Heraklit infrastructure (Fettke & Reisig, 2020).

We evaluate this approach on the Fischertechnik **Agile Production Simulator** (APS)[1], providing the tools for end-to-end prediction. We extract process logs directly from the internal MQTT broker and then fit and train the model based on derived Heraklit steps. The model achieves an outstanding 92% mean accuracy on single next-event prediction. Similar results can still be achieved when simulating noisy MQTT logs by removing events or adding random events into the logs.

TODO: Fill in latest results

The structure of the thesis can be summarized as follows. Chapter 2 introduces the necessary background on Heraklit, Transformers and MQTT while providing information on theorical prediction approach.

Chapter 3 then explains the chosen modelling approach of the Fischertechnik APS case study using Heraklit. Continuing into the technical details, Chapter 4 provides a deeper dive into the details of the applied approach and the Fischertechnik data.

The case study model is lastly evaluated in Chapter 5 in various scenarios, before drawing conclusions, and discussing limitations and future work in Chapter 6.

[1] https://www.fischertechnik.de/de-de/industrie-und-hochschulen/technische-dokumente/simulieren/agile-production-simulation

2

Theory

In this chapter, we will present the theoretical and technical background of this work. This includes defining the Heraklit modeling framework used to model the factory, the process prediction problem and finally the transformer architecture applied to solve it, while providing related work.

2.1 Modelling with Heraklit

Heraklit (Fettke & Reisig, 2020) is a process modelling framework designed to thrive in a discrete digital world, providing a formal foundation for interaction-driven process management of digital and cyber-physical systems.

We will not provide an in-depth explaination of Heraklit, but will focus on an overview of the most important points relevant to this work. In general, Heraklit builds upon three main characteristics:

- **Architecture**: Models can be composed and refined, allowing building large systems using the *composition calculus*.
- **Dynamics**: Actions are performe using local state, and dynamics between actions using causal relationships.
- **Statics**: Items and data and operations on them are treated as first class citizens.

The *composition calculus* of *modules* and causal modelling are what mainly power our approach to process prediction. To understand how they formally work, we will first define the Heraklit notions some of the terms, including **interface** and **module**, **composition of modules** and a **step module**. These definitions are based on definitions found in (Fettke & Reisig, 2020; 2022). Due to the limited scope of the thesis and the limited requirements of Heraklit in our usecase, definitions are not necessarily complete and proofs are left our. They can be read upon in (Fettke & Reisig, 2020).

Heraklit modules are conceptually modelled using graphs, with inner vertices and outer vertices. These outer vertices contribute to the *interface* of a module and are the external connection points of a module.

Definition 1 *Interface*: A *labeled* and *totally ordered* set is an interface.

Definition 2 *Match*: Let A and B be two interfaces, let $a \in A$ and $b \in B$. Then $\{a, b\}$ is a *match of A and B*, if for some label λ, both a and b are $\lambda-$labeled, and

$$|\{a' < a \wedge a' \text{ is } \lambda - \text{labeled} \mid A\}| = |\{b' < b \wedge b' \text{ is } \lambda - \text{labeled} \mid B\}|,$$

ensuring the number of $\lambda-$labeled gates that are smaller than a in A is equal to the number of $\lambda-$labeled gates that are smaller than b in B. A gate of A that does not belong to a match of A and B is *match free* with respect to B.

- Let *matches(A, B)* be the set of all matches of A and B.
- Let *matchfree(A, B)* be the set of all elements of A that do not belong to a match of A and B.

Definition 3 *Module*: A module $M = (V, E)$ is a directed graph, together with two interfaces $*M \subseteq M$ and $M* \subseteq M$ of nodes of M. The interfaces $*M$ and $M*$ are the *left* and *right* interface of M respectively.

The module composition requires composition of graphs along the interfaces. Intuitively, graph composition of two graphs ensures that all graph nodes still exist in the composed graph, just *merging* the nodes at the interface. The new graph thus contains all nodes of both graphs not included in the interface, the free nodes of both interfaces and once the nodes in the interface. One then just needs to reconstruct the edges as before.

Definition 4 *Graph Composition*: Let M and N be two graphs, let $A \subseteq M$ and $B \subseteq N$ be interfaces. Then the *composition of M and N along A and B* is the graph G where:

1. The nodes of G are

   $$(M \setminus A) \cup (N \setminus B) \cup \text{matchfree}(A, B) \cup \text{matchfree}(B, A) \cup \text{match}(A, B)$$

2. For each edge (x, y) of M or N,
   - if x and y are both match free, then (x, y) is an edge of G;
   - if x is match free and $\{y, y'\}$ is a match, then $(x, \{y, y'\})$ is an edge of G;
   - if $\{x, x'\}$ is a match and y is match free, then $(\{x, x'\}, y)$ is an edge of G;
   - if $\{x, x'\}$ and $\{y, y'\}$ are matches, then $(\{x, x'\}, \{y, y'\})$ is an edge of G.

Definition 5 *Module Composition*: Let A and B be two modules. Their composition $A \bullet B$ is defined as follows:

- The graph of $A \bullet B$ is defined as the composition of the graphs (*Def. 4*) of A and B along the interfaces A^* and $^*B$.
- The left interface $^*(A \bullet B)$ of $A \bullet B$ is $^*A \cup \text{matchfree}(^*B, A^*)$. The elements of $^*A$ are ordered before the elements of $\text{matchfree}(^*B, A^*)$.
- The right interface $(A \bullet B)^*$ of $A \bullet B$ is $B^* \cup \text{matchfree}(A^*, {}^*B)$. The elements of B^* are ordered before the elements of $\text{matchfree}(A^*, {}^*B)$.

Module Composition holds two important properties:

- **Associativity**: Let A, B, C be modules. Then $(A \bullet B) \bullet C = A \bullet (B \bullet C)$. This property allows us to ignore the brackets in composition, and merging multiple submodules in arbitrary orders.
- **Commutative** without shared gates: Let A, B be modules. $A \bullet B = B \bullet A$ iff A and B share no equal interface labels. This will be of high interest when composing modules without causal relationships. Their interface would not share any labels, so the order of their composition also does not matter.

Note how in *Def. 3* there is no notion of any dynamics yet. Heraklit follows the idea of petri nets to model the dynamics, so we will now refine our definitions to separate the graph nodes into alternating *places* and *transitions*.

Definition 6 *Net Graph*: Let $G = (V, E)$ be a graph, and let P and T be two disjoint sets with elements called *places* and *transitions*, respectively, such that:

1. $P \cup T = V$;
2. For each edge $(x, y) \in E$ holds: either $x \in P$ and $y \in T$, or $x \in T$ and $y \in P$.

Then G is a *net graph*, and we write $G = (P, T; E)$

Definition 7 *Net Module*: For a net graph G, a module over G is called a *net module*.

Composition of net modules is defined via the composition of modules and produces a valid net module.

To model the discrete stepwise behaviour of processes, we define *step modules*, which only ever include a single *event* and the states it affects. Speaking in terms of net modules, every step module only contains **one transition**.

TODO: Do we really need the “disjoint” requirement? (it should work for our stuff) Additionally, do we want to allow places that are not part of transitions?

Definition 8 *Step Module*: Let $M = (P, \{t\}; E)$ be a net module with disjoint interfaces $^*M$ and M^* with $P = {}^*M \cup M^*$ such that for each $p \in P$ holds: $(p, t) \in E$ iff $p \in {}^*M$, and $(t, p) \in E$ iff $p \in M^*$. Then M is a *step module*.

In the following, we will often refer to *step modules* when describing objects in Heraklit context for simplicity.

Definition 9 *Run Module*: Let $R = (P, T; E)$ be a net module. R is a *run module* iff

1. all places and all transitions are labeled,
2. at each place of R at most one edge begins and at most one edge ends,
3. no edge sequence forms a cycle,
4. $^*R$ contains all places where no edge ends,
5. R^* contains all places where no edge begins.

Important properties of run modules are:
- Each step module is also a run module.
- The composition of run modules generates again a run module.
- Each finite run module R can be composed as $R = P_1 \bullet \ldots \bullet P_n$ from step modules $P_1, \ldots, P_n$.

In the following, *run modules* are often referred to as *runs* for simplicity.

This concludes the most necessary basic Heraklit concepts necessary for the our approach. While Heraklit offers many possiblities to model data, structures and functions, we will not need them for the simple model of the Fischertechnik APS.

Section 2.2.1 shows some graphical examples of step modules and composition into runs. In Chapter 3 the step modules for the Fischertechnik APS are defined, along with some examples of composition.

2.2 Process Prediction

Modern enterprise systems collect huge amounts of data of process executions in so-called **event logs**. The event-logs of just one execution of such a process is called a **trace**. Each event in these logs contains at least an *identifier* to distinctly identfy the process execution, and *event name* describing the action and some sort of *ordering* to express the sequence of events, mostly timestamps and execution times. Additional metadata, such as involved resources, machines or sensoric data, can also be attached to an event.

Process prediction is concerned with predicting **possible outcomes** of ongoing traces. This prediction can be performed online, meaning while the execution of the process happens, such that unwanted outcomes can be prevented by preemptively changing the execution based on the prediction.

Predictions are thus performed on incomplete traces, providing potential outcomes as outputs. The to-be-predicted outcomes are determined by the business problem at hand. They can be broad information about the full process execution such as expected remaining process execution time or whether the process will fail or succeed in its action, or finer-grained information, such as the next possible event or detecting anomalies within events (Neu et al., 2022; Van Der Aalst, 2016).

In this work, we will focus on predicting the next event of a process.

2.2.1 Next-Event Prediction

Given a full execution trace $t = e_1 \rightarrow ... \rightarrow e_n$ of length n and let $p = e_1 \rightarrow ... \rightarrow e_i$ with $i < n$ be a finite prefix of t, the next event might seem to simply be e_{i+1}. This notion is certainly correct in a sense that e_{i+1} is **a** next step of p. However this definition fails to capture the semantics of concurrent or parallel systems, where multiple *correct* linearisations and thus orderings are possible.

We build upon the example from the Introduction:

The process needs to start, then two separate machines produce `A` and `B`, and then a third machine combines them. There are two causal relationships given - `Start` must happen first, and the combination of `A` and `B` must clearly happen after both `A` and `B` were produced. Given the prefix trace `Start`, both `Produce` A and `Produce` B are valid options due to the parallelity given. Both traces

`Start` $\rightarrow$ `Produce` $A \rightarrow$ `Produce` $B \rightarrow$ `Combine` $A \wedge B$

and

`Start` $\rightarrow$ `Produce` $B \rightarrow$ `Produce` $A \rightarrow$ `Combine` $A \wedge B$

can be deemed *correct*. Given the first trace is then later extracted from the system, the second one should still not be deemed incorrect - as we don't want to care about the order of causally unrelated events. Importantly though,

`Start` $\rightarrow$ `Combine` $A \wedge B \rightarrow$ `Produce` $A \rightarrow$ `Produce` B

is **not** a trace we should deem correct.

To define this correctness measure, we use the Heraklit theory. As a first step, we need to create a mapping between events and Heraklit Step Modules. Given this mapping, we can use the following definitions to describe a correct prediction.

> **Definition 10** *Run Prefix*: Let $R = P \bullet S$ be a run module. Then P is a prefix and S a suffix of R.
>
> **Definition 11** *Next Step*: Let R be a run module and P a prefix of R. Then the step module s is a next step after P in R iff $P \bullet s$ is a prefix of R.

Definition 12 *Correct Prediction*: Let R be a run module and P a prefix of R. Then the step module s is a correct prediction of P iff s is a next step after P in R.

Notice how next steps and correct predictions are inherently the same.

We can examine these definitions on our example from above by defining the following steps:

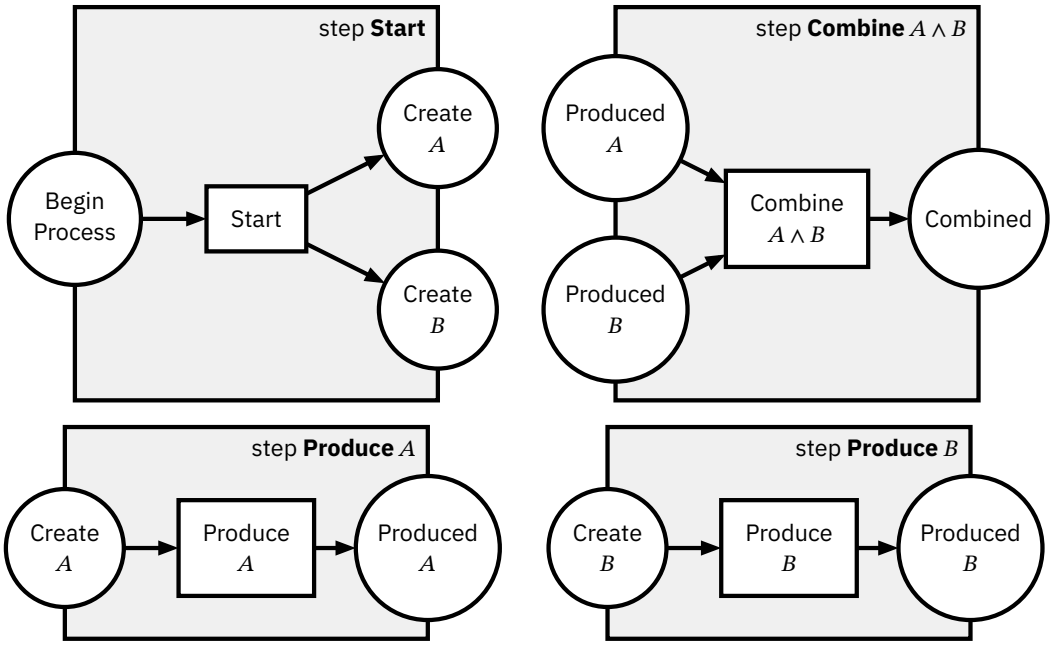


Figure 2.1: *Example Machine Step Modules*

To keep it simple, the labels of the step modules are the same as the labels of the transitions contained within them. The transitions on the left and right on the edge of the module border are the left and right interfaces, respectively. By the layout of the steps one can already grasp as to how valid runs could look.

By our definitions of composition, we already know that

$$\begin{aligned} &\text{Start} \bullet \text{Produce } A \bullet \text{Produce } B \bullet \text{Combine } A \wedge B \\ =\ &\text{Start} \bullet \text{Produce } B \bullet \text{Produce } A \bullet \text{Combine } A \wedge B \end{aligned}$$

This composition can also be shown graphically, as seen in Figure 2.2. By looking at just the first step, **Start**, one can see how both **Produce** A and **Produce** B are correct predictions for such a run, as no causal relationships are existent between them.

To make use of this definition, we first need to model a system by providing the Heraklit step modules. For our case study, they can be found in Chapter 3.

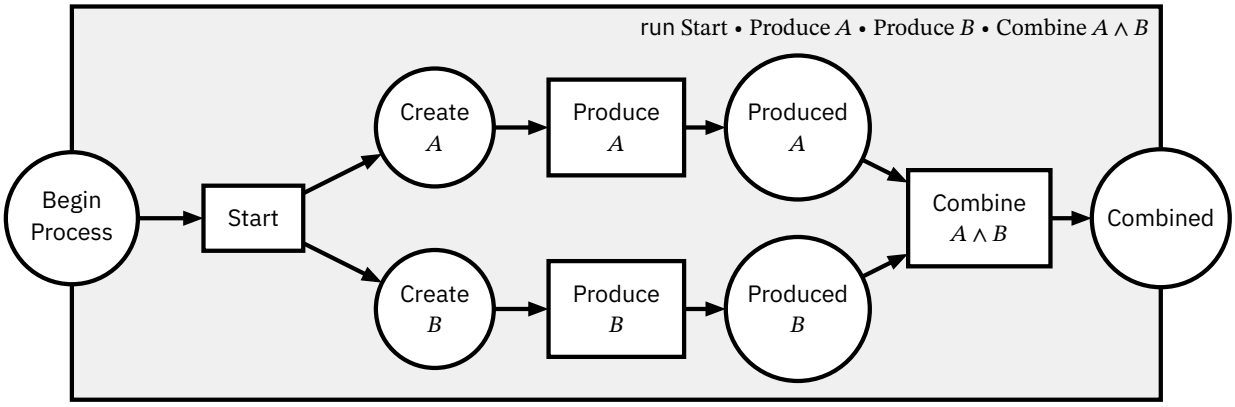


Figure 2.2: *Example Composed Machine Run*

2.2.2 MQTT

MQTT is a client-server protocol using a publish/subscribe pattern. It is considered the most favorable connection pool for Internet of Things (IoT) applications (Yassein et al., 2017).

With MQTT, clients can connect to the server, a *broker*, and *publish* some information to a *topic*. Other clients can connect to the same broker, and *subscribe* to certain topics. Whenever new messages are published to a topic this client is subscribed to, the broker pushes this message to the client. Clients can be *publishers* (or sources) and *subscribers* (or sinks) at the same time. Thus MQTT is a many-to-many communication protocol.

It provides three levels of Quality of Service (QoS), which can deal differently with network performance issues like latency or error rate, at the trade off of network traffic and energy consumption (Toldinas et al., 2019).

1. QoS 0: At most once. The message is sent to all subscribers only once. It is not stored on the broker, and if delivery was not successful, no redistribution is planned. It is also known as *"fire and forget"*.
2. QoS 1: At least once. The message is repeatedly sent to all subscribers that have not acknowledged the message yet, marking every message from the second send on using the `DUP` flag. The message is stored at the broker until all subscribers have received the message.
3. QoS 2: Exactly once. Similarly to QoS 1 the broker stores the message and resends it. However the clients will need to also store the message until it gets released via a special message to ensure no double handling takes place.

The Fischertechnik APS uses MQTT as its main channel of communication between the differnt production modules. While most state updates are distributed via QoS 1, some specific order requests are executed using QoS 2.

Both cases result in duplicate messages from the broker, which need to be filtered out during data processing. The specific MQTT broker can also *retain* messages of QoS 1, which redistributes the latest message from a topic to newly connected clients, even if that message was published before. These messsages need filtering as well, as they can incorrectly influence the assumed state.

2.3 Transformer

Over the last decade, research mostly identified deep-learning approaches as an advancement over traiditional machine learning approaches for predictive process monitoring (Di Francescomarino et al., 2026; Evermann et al., 2017; Mehdiyev et al., 2020; Neu et al., 2022). Especially with a lot of different events requiring high cardinality categorial variables, tradition machine learning approaches start to show their weaknesses (Zhu, 2020).

While using Long Short-Term-Memory models (LSTMs), a special version of recurring neural networks (RNN) showed it successes (Camargo et al., 2019; Tax et al., 2017), later state-of-the-art models use the transformer architecture (Bukhsh et al., 2021; Ni et al., 2023).

2.3.1 Transformer Architecture

First presented in (Vaswani et al., 2017), this deep-learning based model architecture revolutionized its field, with now more than 250.000 direct citations[2]. Originally intended for language translation, it is now most known for the use in Large Language Models (LLMs), that power platforms like ChatGPT (Kulkarni et al., 2023).

The following section will provide a technical overview of the architecture as presented in the original paper. The transformer can generally be split into two parts: The `Encoder` and the `Decoder`, whereas the former focusses on creating an *contextual understanding* of input data and the latter is responsible for *generating output* sequences based on previous output and the understanding of the `Encoder`. Nowadays often only one of the both structures is used, e.g. BERT only uses an `Encoder` layer to learn text representations (Devlin et al., 2019), while GPT and GPT-2 both used an `Decoder`-only architecture (Radford et al., 2018; 2019), as it is focussed on next token generation only.

As our goal of next-event prediction requires us to generate new steps, our model can be a `Decoder`-only network as well. We will therefore focus on presenting the architecture structure that sub-module.

[2] Based on Google Scholar, accessed June 2026

The first step is to convert the input sequence tokens to vectors of size d_{model}. With a context window size d_{ctx}, which is the maximum amount of tokens processed at the same time by the model, this conversion translates our sequence of tokens into a two-dimensional tensor of size $d_{\text{model}} \times d_{\text{ctx}}$. This transformation is performed by a traininable linear layer, essentially the index of the tokens in the vocab to the lower dimension, *embedding* it. Both d_{model} and d_{ctx} are *hyperparameters* of the architecture, as will be further variables written as d_{param}, and need to be chosen before training.

Next follows a *posititional encoding*, where fixed geometrically decreasing frequences are added to the input embeddings to encode the position of the token within the sequence. As no further weight is giving to the explicit original sequence order in the following steps, this encoding provides the only way to distinct two tokens distance in the upcoming layers.

The now properly embedded sequence is now passed through d_{layer} repetitions of the transformer blocks. They each again consist of three sublayers, connected each by a layer normalization. Assuming input x to a sublayer and $\text{Sublayer}(x)$ the function performed by the sublayer, the output is $\text{LayerNorm}(x + \text{SubLayer}(x))$, to keep the output stable without any unexpected outliers complicating the gradient descent during training.

Two of the sublayers are *Multi-Head Attention Layers*. Here the current embedding is first multiplied by $d_h \cdot 3$ linearly learnable parameter matrices $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_i^V \mathbb{R}^{d_{\text{model}} \times d_v}$ with $1 \leq i \leq d_h$. The resuls are triples of the form (Q_i, K_i, V_i). In more optimal implementations, this computation does not need to perform $3 \cdot d_h$ matrix multiplications, but instead combined into one larger matrix multiplication. The triples are fed into the *scaled dot-product attention mechanism*, defined as

$$\text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{\text{mask}(Q_i K_i^T)}{\sqrt{d_k}}\right) V_i$$

This computation can be intuitively understood as follows.

1. Combining Q_i and K_i: Q_i represents a certain learned query, essentially encoding which part of the embedding is interested in getting certain information of other tokens. K_i highlights positions in the embedding, that are match this information of the query Q_i. By multiplying them, one combines the interested embedding with the tokens that contain information.
2. Lowering the magnitude of the values in the combined matrix by dividing by $\sqrt{d_k}$, such that the softmax function has smoother gradients.
3. Masking out all fields that try to let tokens refer to tokens after them, by setting the matrix upper right diagonal to $-\infty$.

4. $\mathrm{softmax}$ itself performs a rowwise normalisation, such that all rows represent a random distribution, i. e. summing up to 1 while keeping the relative magnitude information. Values of $-\infty$ result in 0.
5. Multiplying by V_i: V_i contains the embedding of the information that is present, if the query and key match. Thus the multiplication linearly scales that information within the embedding.

The resulting matrices are of the shape $d_{\text{ctx}} \times d_v$. They are now concatinated into one $h \cdot d_{\text{ctx}} \times d_v$ matrix and multiplied by a last parameter matrix $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

The third sublayer of the transformer block is a simple 2-layer fully-connected feed-forward network with a ReLU activation function. The input and output layers have dimension d_{model} and the inner layer $d_{\text{dim\_ff}}$.

After the transformer blocks, we need to extract the next token. Similarly to the initial embedding, we now need to map the embedded d_{ctx} d_{model}-dimensional vectors back to our tokens. We again apply a learnable linear layer to the embedding, resulting in vectors the same size as the vocab with all tokens. After normalisation, the vector at position i contains a probability distribution over the token at position $i + 1$.

We acknowledge that there are further optimizations to this architecture since the original release, mostly on performance and resource usage (Kwon et al., 2023), and that there are multiple adaptions to other domains such as image processing (Han et al., 2021). Due to our limited dataset and resulting small parameter set, model performance and resource are not a concern for us.

2.3.2 Model Training

Training not only consists of fitting our parameters to the training token sequences. Before training, we need to perform a hyperparameter selection.

Besides the model parameters from above, we need to also select parameters for the deep learning optimizer, in this case the AdamW optimizer (Loshchilov & Hutter, 2019), which builds upon the Adam optimizer (Kingma & Ba, 2017), improving its generalization performance by decoupling the weight decay. Parameterwise it takes a learning rate, the weight decay, two running average parameters β_1, β_2 and a numerical stability parameter ε.

Using cross validation on k-fold splits of the training data, we search through a subset of hyperparameters. These hyperparameters are based on the model defaults and changed according to the training data size we provide.

2.3.3 Technical requirements for Process Prediction

Some approaches to apply the transformer architecture rely on natural language to express the processes steps (Rebmann et al., 2025). However, this approach requires the model not only learning things about the process, but also understanding the language.

Our approach to use the transformer architecture therefore relies on training a *smaller*[3] model following the original transformer architecture from scratch.

The architecture requires us to encode our Heraklit steps into *tokens*. We chose to make every possible step its own token, as our case study does not require taking in parameters for steps. In case of parameters, one can simply encode them as a sequence of tokens, or by creating multi-dimensional input- and output vectors that are parsed as parameters. The output of the model includes the *probabilities* of the different next tokens. One can either just choose the one with the highest probability, or sample off of the then provided distribution.

Further details on Fischertechnik APS specifics and the training of our model are discussed in Chapter 4.

[3]Compared by the number of learnable parameters to a typical LLM

3

Modelling

This section marks the beginning of our case study, showcasing the approach of using Heraklit and the Transformer architecture to perform next-step process prediction.

We want to evaluate our process prediction technique on the Fischertechnik **Agile Production Simulator** (Fischertechnik, n.d.). As required by our definition of what a correct prediction is (*Def. 12*), we will first need to translate events of the APS into Heraklit (Fettke & Reisig, 2020) steps. These steps should crucially allow the modelling of concurrency within the system by only modelling the causal relationships of events.

While some domain knowledge is necessary for this step, still only step modules need to be created, not full system process models. As these steps are predicted by our technique, having a clear understanding of what each step represents is useful during further evaluation.

We can split and group the steps by the relevant factory modules:

- Automated Guided Vehicle (**AGV**): It autonomously transports workpieces between factory modules.
- High-Bay Warehouse (**HBW**): It serves as a storage system to the factory, designed with an automatic retrieval.
- Delivery And Pickup Station (**DPS**): It serves as the point of input and output of the factory. New workpieces are *inserted* here, processed workpieces are *shipped off*. The included NFC reader starts the tracking of workpieces across the factory.
- Drilling Station (**DRILL**): It performs a simulated drilling operation on the workpieces, picking them up from and dropping them onto the AGV.
- Milling Station (**MILL**): It performs a simulated milling operation on the workpieces, picking them up from and dropping them onto the AGV.
- Quality Control with AI (**AIQS**): Checks the quality of a workpiece using a camera and color sensor. If a failure occurred - represented by a erroneous print on the workpiece - the workpiece is discarded.

- Central Control Unit (**CCU**): It is the central controller of the factory. While it has no physical representation in the factory, it is the centralized decision maker of the factory, synchronizing the different distributed modules. Thus for our modelling purposes, it will be the glue that connects the modules together.

3.1 Heraklit Step Modelling

3.1.1 AGV

We start by modelling the AGV. It is the main source of interaction in the APS, however it can only perform one action by itself, which is moving from one processing module to the next. As we want to have one token per step, we need to create multiple **move AGV** steps.

For each module `M1` $\in \{\text{DPS}, \text{HBW}, \text{DRILL}, \text{MILL}, \text{AIQS}\} := M$ we need to have a move step to each module `M2` $\in M \setminus \{m\}$. Thus in the following step module template, we get all possible **move AGV** steps by instantiating `M1` and `M2` with all possible combinations.

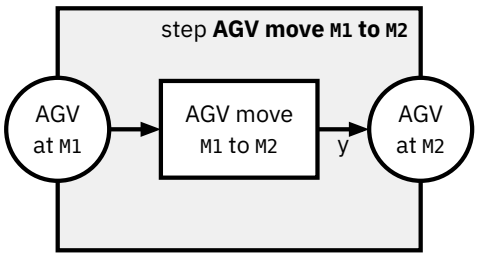


Figure 3.1: *AGV step template*

A keen reader with might realize that this could be modeled using a parameterized module, an advanced Heraklit concept not introduced in Chapter 2. As no further parameterization was necessary in the remaining modelling, the introduction, definitions and complexity can be reduced by showcasing the template steps instead. For the sake of completeness, the step module as a parameterized module is shown below.

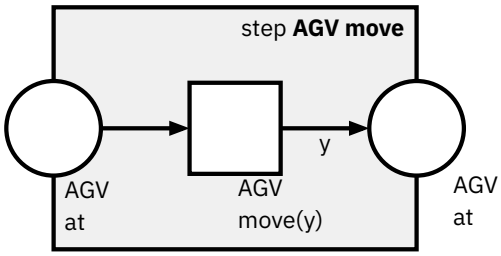


Figure 3.2: *Parameterized AGV step*

This type of modelling requires defining variables and domains as

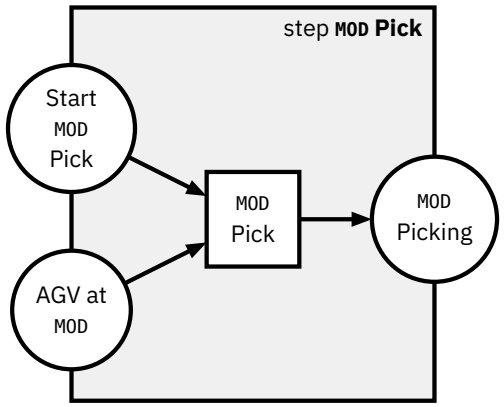
```
variable
y: process-module

domain
process-module: {DPS, HBW, DRILL, MILL, AIQS}
```

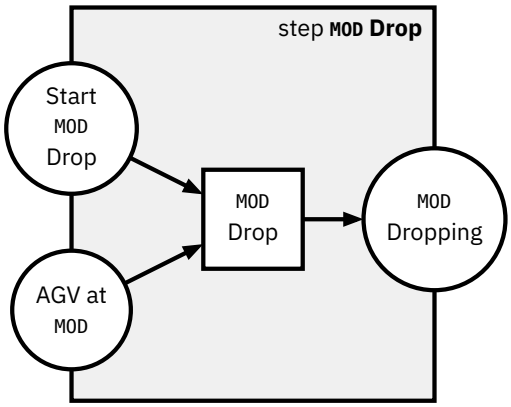
Important to notice is that depending on which **AGV move** step is taken, different step modules depending on the AGV position can be composed afterwards. This however also ensures that modules can depend on the AGV being at their module, locking them at that position by temporarily consuming the token at the `AGV at MOD` place.

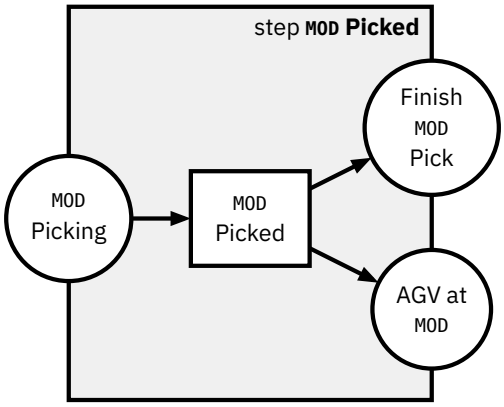
3.1.2 Picking and Dropping

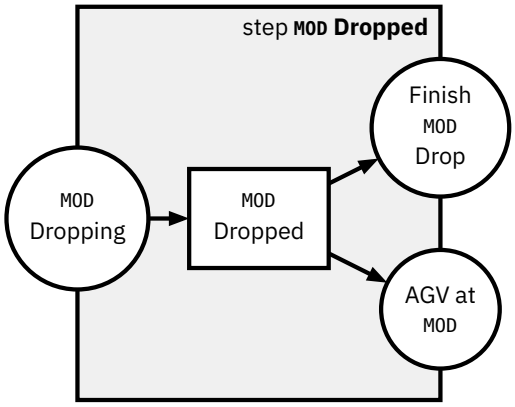
All other modules need to phsycially interact with the remaining factory by picking up workpieces from the AGV or dropping workpieces onto the AGV. This workflow is generally the same over all modules, thus to reduce wasted space, we again fall back to modelling these steps using the following template, with

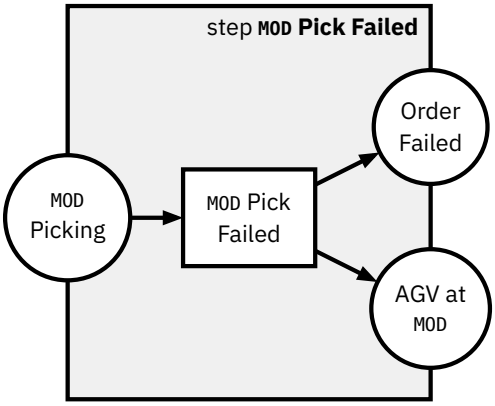
`MOD` $\in \{\text{DPS}, \text{HBW}, \text{DRILL}, \text{MILL}, \text{AIQS}\}$











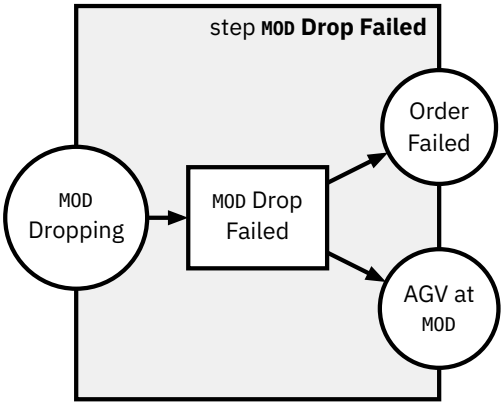


Figure 3.3: *Pick steps template*

Figure 3.4: *Drop steps template*

Two points should be highlighted:

1. We are modelling failure modes. In case the action fails, the respective `Pick Failed` or `Drop Failed` can be composed instead of the `Picked` or `Dropped` successful counterpart.
2. The AGV is not able to move while the picking or dropping action is performed, as it consumes the token at `AGV at MOD` token.

Next we will look at the individual process module actions.

3.1.3 HBW

The High-Bay Warehouse actions only consist of picking up workpieces from the AGV and putting them into storage, and of dropping workpieces from storage onto the AGV.

This might seem like actions that require a lot of control, first due to needing to select what workpiece should be extracted from storage, and second due to storage management itself.

However, the Fischertechnik simplified both control challenges by

1. only looking at the current running order that the pick or drop action is associated with, figuring out what color it refers to and then
2. performing a *First-In First-Out* (FIFO) storage policy for all colors, keeping the mapping of colors to ordered storage slots persistent.

Thus when executing a pick or drop, it can do so without any further information. We decide to not try to model the internal (unexposed) storage logic here, as no events are emitted through the MQTT logs and therefore no step can be mapped to it. The generic `MOD Pick` and `MOD Drop` actions defined in Figure 3.3 and Figure 3.4 can be reused here.

A successful HBW pick run can be seen in Figure 3.5, a failed HBW pick Figure 3.6.

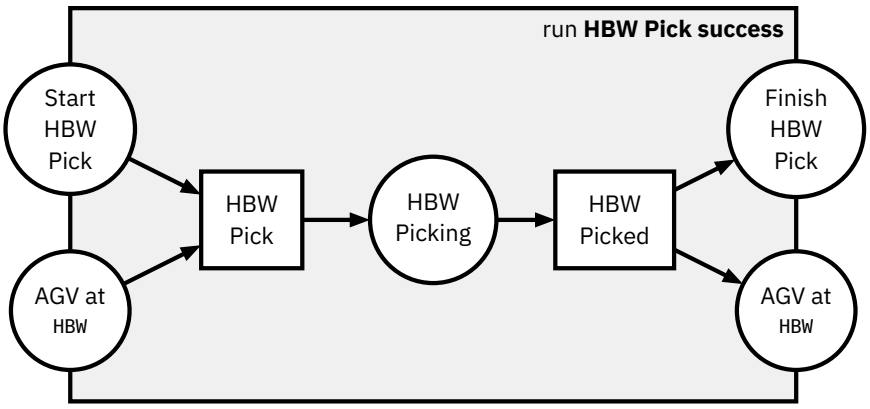


Figure 3.5: *HBW Pick run (Store into HBW)*

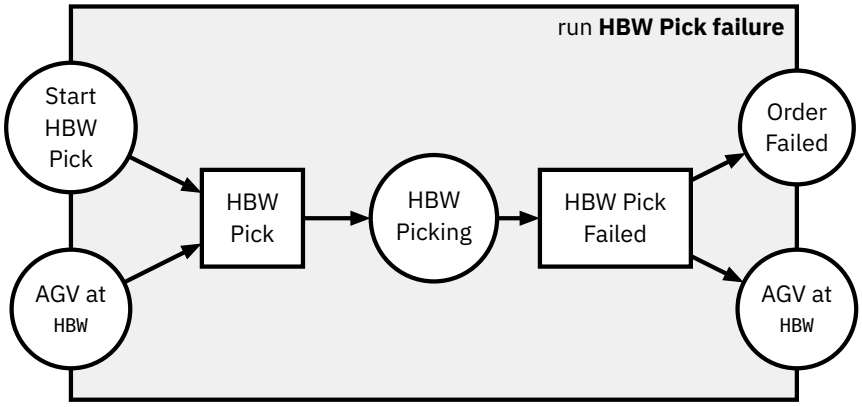


Figure 3.6: *HBW Pick run failure (Store into HBW)*

They are clearly defined by composing the singular pick steps:

$$\text{HBW Pick success} = \text{HBW Pick} \bullet \text{HBW Picked}$$
$$\text{HBW Pick failure} = \text{HBW Pick} \bullet \text{HBW Pick Failed}$$

These runs are exemplary for all module `Pick` and `Drop` runs, as they always follow the same structure.

3.1.4 DPS

The DPS is responsible for insertion of pieces into the factory and shipping them out of the factory. The piece insertion into the factory is represented by a `DPS Drop`, dropping the workpiece onto the AGV from the loading bay. The shipping out of the factory by `DPS Pick`, picking the workpiece up from the AGV and dropping it off at the loading bay.

Both actions are reading and writing to the NFC tag on the workpiece, if present, providing a history of the piece across the factory. Similarly to the physical actuator control done by the Siemens SPS, we choose to not model the NFC tag explicitly, as the prediction will be on a higher level of abstraction. Thus, we can again re-use the template for Picking and Dropping given in the templates in Figure 3.3 and Figure 3.4.

Notably, the `DPS Drop` step is the only step that can introduce a new workpiece into the factory. New workpieces are generally first stored into the HBW and then extracted again with an order, even if an order for that piece is already present.

The failure case of a `DPS Drop` could require different processing than other dropping steps, as this is the only action where a workpiece might not have an order associated to it yet. However, as we are only processing single workpieces at a time in our dataset, the order failure case is an appropriate failure model, as either way the processing of that workpiece ends there.

We do not need to introduce any additional new steps, as again the abstraction given through the MQTT traces does not provide further details.

3.1.5 DRILL

Like all other physical modules, the drill process module has to interact with the AGV to process workpieces. Therefore the `Pick` and `Drop` template is defined for the it as well.

Additionally, this is the first module to perform a certain module-specific action observable via the MQTT logs. This unique action is simulated drilling of a hole into the workpiece, which can again succeed or fail. The matching Heraklit step definitions can be seen in Figure 3.7. Notice that

1. We require a workpiece to be picked up from the AGV to be able to start the drill action by consuming the token at the place `Finish DRILL Pick`.
2. We do not require the AGV to be at the drill module during the drilling steps, opening up the future possiblity of having multiple modules process workpieces simultaneously within the same model, as the AGV can drive to a different station during processing at the drill.

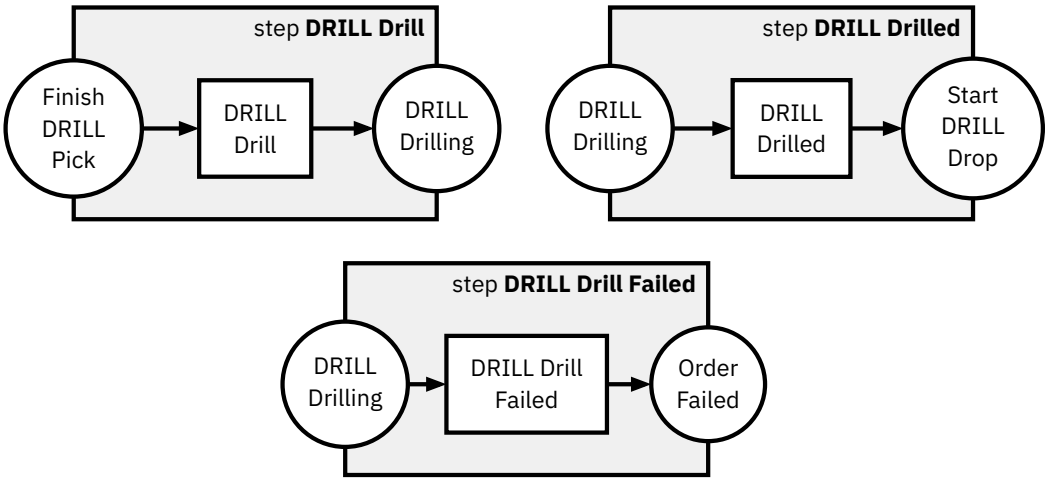


Figure 3.7: *DRILL Drill steps*

Both of these properties can be re-discovered in the following module-specific step-definitions.

While there is a parameter for the drilling duration defined for each workpiece color, we decide to not model it due to two reasons. First, the parameter is constant and thus just belongs directly to the drilling operation of that color. Second, even assuming it was not constant, there is no causal relationship to be learned as to why the duration should be different. The Fischertechnik APS provides no simulated tool wear or different operation durations for simualted broken or working workpieces.

We again provide a successful run in Figure 3.8 and a failed run in Figure 3.9, composed of:

$$\text{DRILL Drill success} = \text{DRILL Drill} \bullet \text{DRILL Drilled}$$
$$\text{DRILL Drill failure} = \text{DRILL Drill} \bullet \text{DRILL Drill Failed}$$

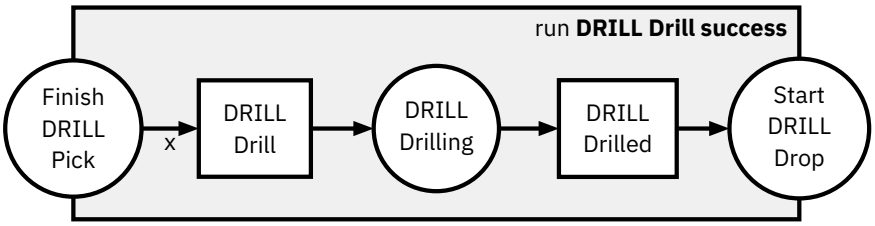


Figure 3.8: *DRILL Perform Drilling run*

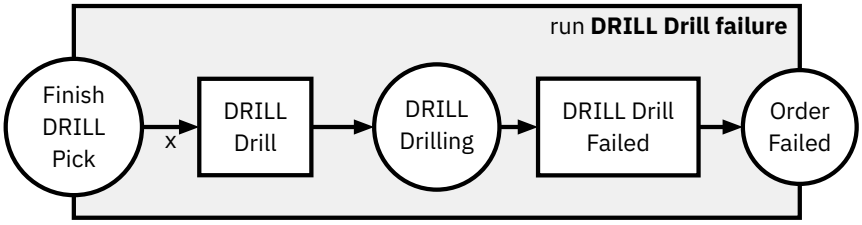


Figure 3.9: *DRILL Drilling Failure run*

3.1.6 MILL

Similarly to the drill, the mill process module performs a simulated action, but it is an operation to simulate milling a grove into the workpiece instead of the drilling operation. It again also contains the template `Pick` and `Drop` steps.

Thus the steps are defined identically modulo renaming in Figure 3.10. Thus the same properties hold and the runs can be composed in a similar fashion as in Figure 3.8 and Figure 3.9.

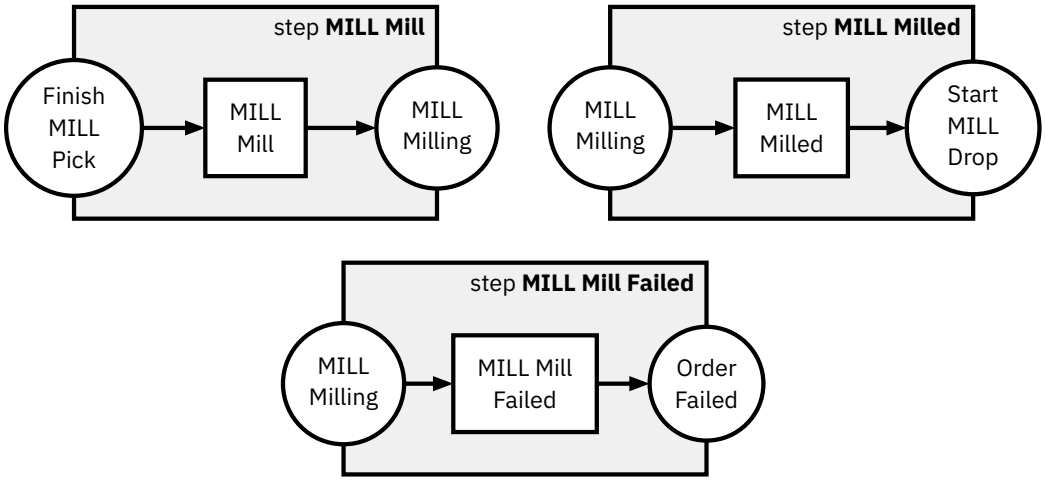


Figure 3.10: *MILL Mill steps*

3.1.7 AIQS

The last physical module is the AIQS, which provides a quality control checkpoint for all orders. To simulate failure within the APS, workpieces inserted into the factory at the DPS are designated `fail` or `pass` pieces, determined by a print on top of the piece. Using a camera and a color sensor, it can detect these faults and then discard faulty pieces into a trash chute, while passing good pieces on.

Besides the `Pick` and `Drop` steps, the AIQS needs to perform this quality check action. The matching steps are defined in Figure 3.11.

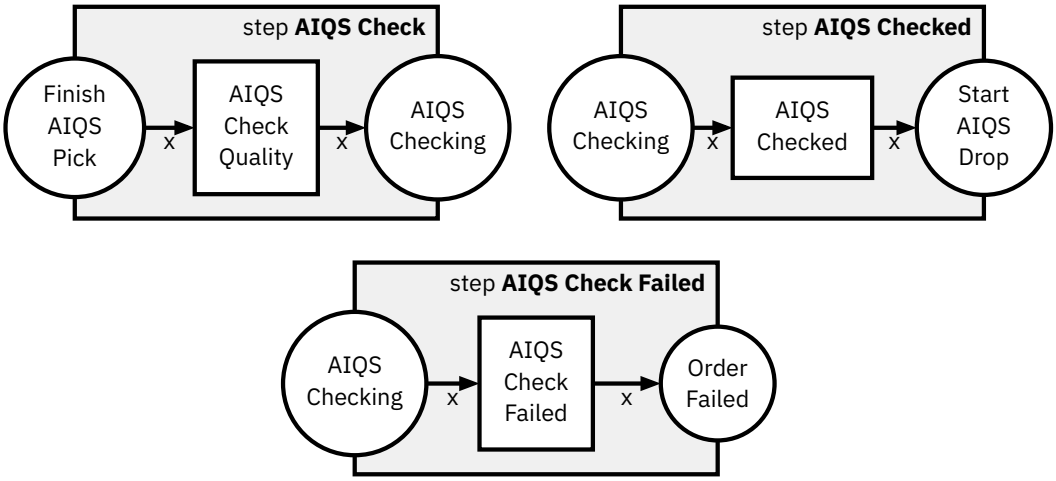


Figure 3.11: *AIQS Check Quality steps*

In the Fischertechnik APS, only the AIQS is concerned with the failure of a piece, all other pieces are processed based on just the color. This makes the behaviour of the next step after a started quality check hard to predict: The piece must either fail or pass, but the previous process execution provides no indication of whether the workpiece "processed" will result in a failure or not. This is a shortcoming in the simulation of the APS, as especially tool wear and processing durations could be exploited to simulate a failing processing module on a piece, thus that a prediction has some grounds to base its decision on.

We will later see that this results in missed accuracy of our prediction model.

A composed success run of the AIQS can be seen in Figure 3.12, the failure in Figure 3.13.

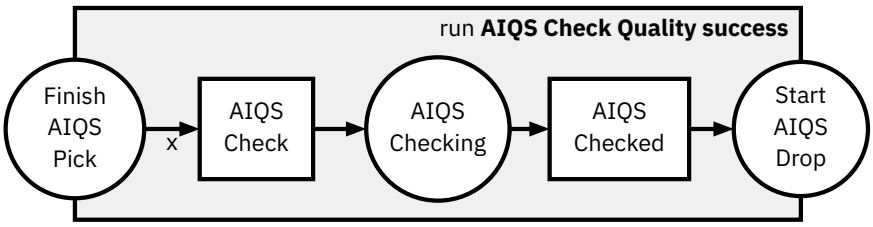


Figure 3.12: *AIQS Check Quality run*

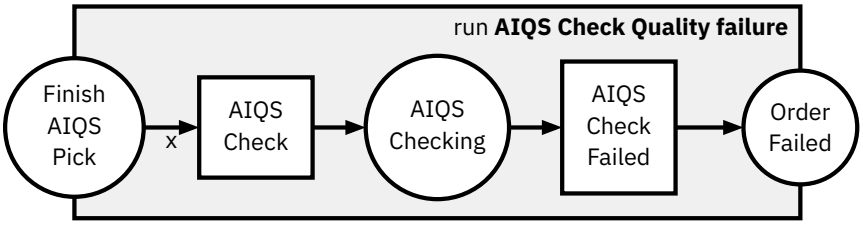


Figure 3.13: *AIQS Check Quality failed run*

3.1.8 CCU

The CCU is the heart of the factory. It controls the interactions between the modules, taking the decisions on where the AGV should go and interacting with the factory order system via the UI.

Our goal is to model the steps extractable from the Fischertechnik MQTT Logs. These control steps, deciding what module action happens after the next, is **implicit** or **invisble** control. There is no control action token to be found in the MQTT logs that clearly defines *after action X perform action Y*, the control can just be inferred from the interactions between the modules. This means that the CCU steps will and can not be present in the prediction tokens, as they do not exist within the logs. However, we still need to model some control system, as the steps of different modules are not composable at the moment.

For a potential synthetic generation of runs modelling the different variations of runs, e.g. depending on the color of the workpiece, one could argue that these control steps must be meticulously designed, including the order of stations per workpiece type. This would imply pre-modelling a specific order of module actions into the steps via the connecting places. An example can be seen in Figure 3.14. Here we provide the fixed connection of a `MILL` step to the `DRILL` step.

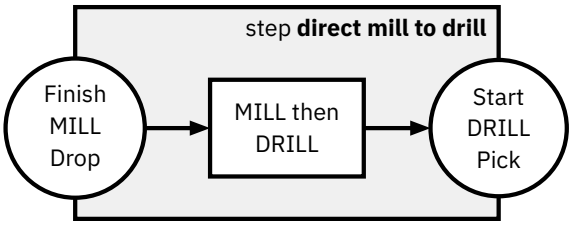


Figure 3.14: *Direct Module Interaction Modelling Approach*

This approach has multiple downsides. We trade the increased detail for **decreased flexibility**, as changes in the configuration for certain workpieces would require a new Heraklit step model. More importantly though, the tools to validate model outputs would need to be capable of handling an **exponential number of steps** to support all different process configurations, with increases in modules, and again expontential growth with length of runs. This is due to these control steps not being present within the logs, so all matching control steps must be

tried to be appended at any point of the validation, creating a huge search tree for validation.

We therefore decide not to model all the configurations explicitly. Instead, we only restrict our model to allow one module action to take place at a time. While this might seem counter-intuitive when looking at the distributed factory setting, here we are only looking at the factory execution from the perspective of a singular workpiece. Since all processing parts of a singular workpiece are always only present in one processing module's action, these actions don't need to be able to run concurrently.

Technically, this restriction is applied by creating a global shared place called `Next Module Ready`. Whenever a module wants to start, it needs to consume this place, whenever it is finished, it will fill the place again. Following the example from before, this creates the new steps in Figure 3.15.

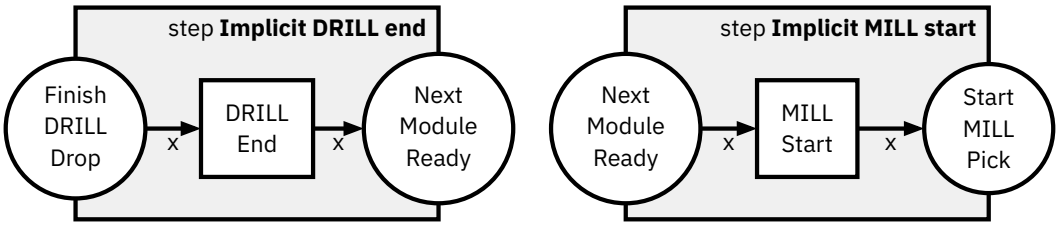


Figure 3.15: *Implicit Control of Mill to Drill steps*

This new design does not directly solve the issue of these control steps missing in the logs, but provides a simple solution. By composing DRILL Dropped • Implicit DRILL end and Implicit MILL start • MILL Pick we create a run module that essentially just changes the interfaces of the module steps to have the `Next Module Ready` place instead of their respective `Start` and `Finish` places. These newly composed models can be then again composed directly via the `Next Module Ready`.

As we know that before and after all module actions their steps will need their one matching `Implicit` step, we can pre-compose the control and module steps when talking about the actual logs.

For example, if the logs contain the steps

`DRILL Dropped` → `MILL Pick`

we can instead interpret this as

`DRILL Dropped` → `Implicit DRILL end` → `Implicit MILL start` → `MILL Pick`

as the implicit steps can be directly inferred.

At this point, one might wonder why we have not directly put the `Next Module Ready` step into the module steps. The reason for this decision is that we can keep

the level of detail on the module basis, while essentially just having an interface wrapper to simplify implementation details later.

All the implicit control steps, that are are implicitely composed with the respective `Start` and `Finish` steps of the processing modules can be found in Figure 3.16. Notably, the DRILL, MILL and AIQS only connect a start module on the `Pick` action and a stop module on the `Drop` action. The DPS and HBW however can independently Pick and Drop without any ongoing action in between, so both `Pick` and `Drop` actions have their own implicit start and stop control.

For the further processing we then also redefine the following step modules as runs composed with their implicit control step:

DRILL Pick ≔ DRILL Start • DRILL Pick

DRILL Drop ≔ DRILL Drop • DRILL End

MILL Pick ≔ MILL Start • MILL Pick

MILL Drop ≔ MILL Drop • MILL End

AIQS Pick ≔ AIQS Start • AIQS Pick

AIQS Drop ≔ AIQS Drop • AIQS End

DPS Pick ≔ DPS Pick Start • DPS Pick

DPS Picked ≔ DPS Picked • DPS Pick End

DPS Drop ≔ DPS Drop Start • DPS Drop

DPS Dropped ≔ DPS Dropped • DPS Drop End

HBW Pick ≔ HBW Pick Start • HBW Pick

HBW Picked ≔ HBW Picked • HBW Pick End

HBW Drop ≔ HBW Drop Start • HBW Drop

HBW Dropped ≔ HBW Dropped • HBW Drop End

Since runs can be composed the same way individual step modules can, we do not require further differentiation and can **assume from now on, that the starting and stopping "steps" can be composed via the `Next Module Ready` place**.

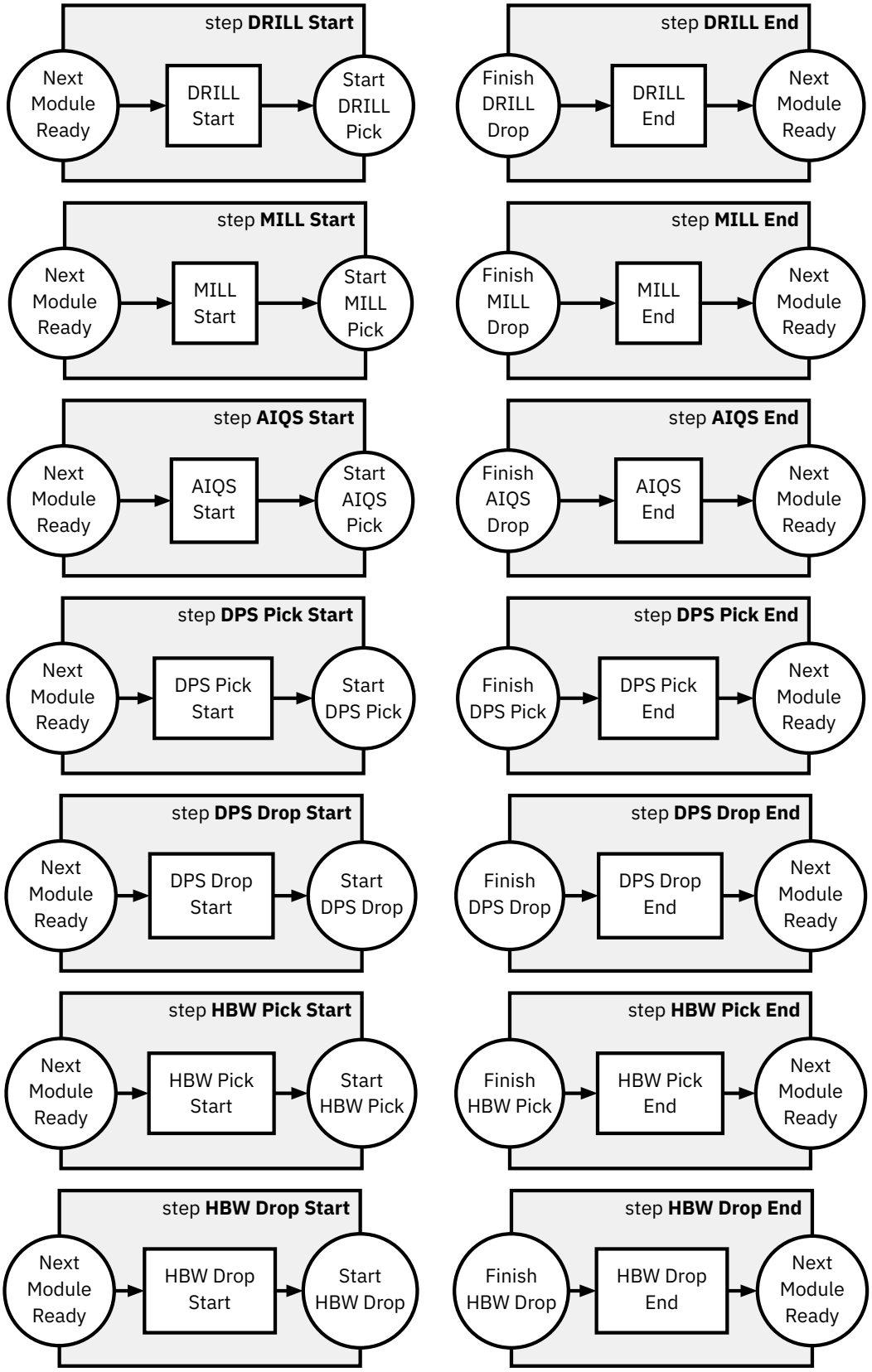


Figure 3.16: *Implicit Control steps*

4

Implementation

This chapter provides the technical details of the implementation of our process prediction technique with encompassing information on the data collection, preprocessing and on the architecture and training of the final model.

While the chapter will describe what we do and how we do it, it explicitly does not explain how to technically execute the code infrastructure. This information is available through a collection of *Markdown files* that provide a walkthrough the project code, including the commands to execute in the command line.

For our implementation we rely on several tools:

- `python`[4]: The programming language used for the code infrastructure and scripts.
- `uv`[5]: A Python project and package manager handling the other listed dependencies and tools.
- `pytorch`[6]: An optimized python library for deep learning. It supports training on CPUs and GPUs, including integrated GPUs such as ones used in Apple Silicon processors. We use it to build our models by using integrated base models or layers, and to perform the training.
- `numpy`[7]: An optimized python library for array programming, such as tensors. This is also implicitely used by `pytorch`, and we use it to interact with data to and from `pytorch`.
- `graphviz`[8]: A library for a graph drawing software for python. We use it to visualize Heraklit runs.

The implementation was tested on both *macOS 15.7* and *Ubuntu Linux 24.04*. The hardware used for training and evaluation consists of an *Apple MacBook Pro* with an *M1 Pro* chip and *32GB* of memory.

[4] https://www.python.org *last accessed: 11.06.2026*
[5] https://docs.astral.sh/uv/ *last accessed: 11.06.2026*
[6] https://docs.pytorch.org/docs/2.12/index.html *last accessed: 11.06.2026*
[7] https://numpy.org *last accessed: 11.06.2026*
[8] https://graphviz.readthedocs.io/en/stable/ *last accessed: 11.06.2026*

4.1 Data Collection and Preprocessing

- split is valid, as all exeuctions are independent of another
- want to extract all the steps required for the work piece

4.2 Model Architecture

-> embedder

4.3 Training Details

training: Adam optimizer, regularization using dropout.

time to train

5

Evaluation

In this chapter we will first compare our prediction model with a random baseline, showing that our model is able to learn about the underlying process. We then evaluate different generalization capabilities of our model.

- Generalization

-> (Pfeiffer et al., 2025)

TODO: Describe further evaluation...

- Time required for potential online use
- Caveats

6

Conclusion

TODO: Write conclusion. Repeat problem statement, then focus on results and implications

6.1 Limitations and Future Work

Bibliography

Aalst, W. van der. (2016). Data Science in Action. In *Process Mining: Data Science in Action* (pp. 3–23). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-49851-4_1

Bukhsh, Z. A., Saeed, A., & Dijkman, R. M. (2021). ProcessTransformer: Predictive Business Process Monitoring with Transformer Network. *Corr*. https://arxiv.org/abs/2104.00721

Camargo, M., Dumas, M., & González-Rojas, O. (2019). Learning accurate LSTM models of business processes. *International Conference on Business Process Management*, 286–302.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186.

Di Francescomarino, C., Donadello, I., & Maggi, F. M. (2026). *Predictive Process Monitoring*. Springer.

Evermann, J., Rehse, J.-R., & Fettke, P. (2017). Predicting process behaviour using deep learning. *Decision Support Systems*, *100*, 129–140.

Fettke, P., & Reisig, W. (2020). Modelling service-oriented systems and cloud services with Heraklit. *Corr*. https://arxiv.org/abs/2009.14040

Fettke, P., & Reisig, W. (2022). Modularization, Composition, and Hierarchization of Petri Nets with Heraklit. *Corr*. https://arxiv.org/abs/2202.01830

Fischertechnik. (n.d.). *Fischertechnik Agile Production Simulation*. Retrieved June 11, 2026, from https://www.fischertechnik.de/de-de/industrie-und-hochschulen/technische-dokumente/simulieren/agile-production-simulation#uebersicht

Han, K., Xiao, A., Wu, E., Guo, J., XU, C., & Wang, Y. (2021). Transformer in Transformer. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, & J. W. Vaughan (Eds.), *Advances in Neural Information Processing Systems: Vol. 34. Advances in Neural Information Processing Systems*. https://proceedings.neurips.cc/paper_files/paper/2021/file/854d9fca60b4bd07f9bb215d59ef5561-Paper.pdf

Kingma, D. P., & Ba, J. (2017,). *Adam: A Method for Stochastic Optimization*. https://arxiv.org/abs/1412.6980

Kulkarni, A., Shivananda, A., Kulkarni, A., & Gudivada, D. (2023). The ChatGPT Architecture: An In-Depth Exploration of OpenAI's Conversational Language Model. In *Applied Generative AI for Beginners: Practical Knowledge on Diffusion Models, ChatGPT, and Other LLMs* (pp. 55–77). Apress. https://doi.org/10.1007/978-1-4842-9994-4_4

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., & Stoica, I. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 611–626. https://doi.org/10.1145/3600006.3613165

Loshchilov, I., & Hutter, F. (2019,). *Decoupled Weight Decay Regularization*. https://arxiv.org/abs/1711.05101

Mehdiyev, N., Evermann, J., & Fettke, P. (2020). A Novel Business Process Prediction Model Using a Deep Learning Method. *Business & Information Systems Engineering*, *62*(2), 143–157. https://doi.org/10.1007/s12599-018-0551-3

Muehlen, M. z., & Ho, D. T.-Y. (2006). Risk Management in the BPM Lifecycle. In C. J. Bussler & A. Haller (Eds.), *Business Process Management Workshops: Business Process Management Workshops*.

Neu, D. A., Lahann, J., & Fettke, P. (2022). A systematic literature review on state-of-the-art deep learning methods for process prediction. *Artificial Intelligence Review*, *55*(2), 801–827. https://doi.org/10.1007/s10462-021-09960-8

Ni, W., Zhao, G., Liu, T., Zeng, Q., & Xu, X. (2023). Predictive Business Process Monitoring Approach Based on Hierarchical Transformer. *Electronics*, *12*(6). https://doi.org/10.3390/electronics12061273

Pfeiffer, P., Abb, L., Fettke, P., & Rehse, J.-R. (2025). Learning from the Data to Predict the Process: P. Pfeiffer et al. *Business & Information Systems Engineering*, *67*(3), 357–383.

Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., & others. (2018). *Improving language understanding by generative pre-training*.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., & others. (2019). Language models are unsupervised multitask learners. *Openai Blog*, *1*(8), 9.

Rebmann, A., Schmidt, F. D., Glavaš, G., & Aa, H. van der. (2025). On the potential of large language models to solve semantics-aware process mining tasks. *Process Science*, *2*(1), 10.

Tax, N., Verenich, I., La Rosa, M., & Dumas, M. (2017). Predictive Business Process Monitoring with LSTM Neural Networks. In E. Dubois & K. Pohl (Eds.), *Advanced Information Systems Engineering: Advanced Information Systems Engineering*.

Toldinas, J., Lozinskis, B., Baranauskas, E., & Dobrovolskis, A. (2019). MQTT Quality of Service versus Energy Consumption. *2019 23rd International Conference Electronics*, , 1–4. https://doi.org/10.1109/ELECTRONICS.2019.8765692

Van Der Aalst, W. (2016). Data science in action. In *Process mining: Data science in action: Process mining: Data science in action* (pp. 3–23). Springer.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*.

Yassein, M. B., Shatnawi, M. Q., Aljwarneh, S., & Al-Hatmi, R. (2017). Internet of Things: Survey and open issues of MQTT protocol. *2017 International Conference on Engineering & MIS (ICEMIS)*, , 1–6. https://doi.org/10.1109/ICEMIS.2017.8273112

Zhu, T. (2020). Analysis on the Applicability of the Random Forest. *Journal of Physics: Conference Series*, *1607*(1), 12123. https://doi.org/10.1088/1742-6596/1607/1/012123